

Лабораторная работа №3

Партиционирование PostgreSQL: разделение больших таблиц и автоматизация создания партиций

1. Цель работы

В этой лабораторной работе необходимо научиться:

- создавать партиционированные таблицы PostgreSQL;
 - работать с разными стратегиями партиционирования;
 - выбирать подходящий ключ партиционирования;
 - понимать `partition pruning`;
 - использовать индексы внутри партиционированных таблиц;
 - анализировать планы выполнения запросов;
 - автоматически создавать новые партиции;
 - контролировать наличие будущих партиций;
 - обнаруживать сбой автоматического создания партиций;
 - отправлять уведомление об обнаруженной проблеме;
 - применять партиционирование к собственной базе данных.
-

2. Теоретическая часть

В предыдущей лабораторной работе мы рассмотрели ситуацию:

данных становится всё больше, и одного индекса может быть недостаточно.

Теперь рассмотрим следующий уровень.

Пусть таблица содержит:

100 млн записей
500 млн записей
1 млрд записей
2 млрд записей

При этом данные имеют естественную структуру.

Например:

events
├─ данные за январь
└─ данные за февраль

```
|— данные за март
|— ...
```

или:

```
orders
|— customer_type = B2C
|— customer_type = B2B
|— customer_type = Enterprise
```

или:

```
transactions
|— hash bucket 0
|— hash bucket 1
|— hash bucket 2
|— ...
```

В таких случаях таблицу можно разделить на несколько физических частей — **партиций**.

Для приложения при этом таблица может выглядеть как одна:

events

а PostgreSQL внутри работает с:

```
events_2026_01
events_2026_02
events_2026_03
...
```

3. Часть 1. RANGE PARTITIONING по дате

Начнём с наиболее распространённого сценария.

Создайте таблицу:

```
CREATE TABLE events (
  id BIGINT NOT NULL,
  user_id BIGINT NOT NULL,
  event_type VARCHAR(50) NOT NULL,
  payload TEXT,
  created_at TIMESTAMP NOT NULL
) PARTITION BY RANGE (created_at);
```

Создайте три партиции:

```
CREATE TABLE events_2026_09_09
PARTITION OF events
FOR VALUES FROM ('2026-09-09') TO ('2026-09-10');

CREATE TABLE events_2026_09_10
```

```
PARTITION OF events
FOR VALUES FROM ('2026-09-10') TO ('2026-09-11');
```

```
CREATE TABLE events_2026_09_11
PARTITION OF events
FOR VALUES FROM ('2026-09-11') TO ('2026-09-12');
```

Заполните таблицу тестовыми данными.

Проверьте:

```
SELECT
    tableoid::regclass AS partition_name,
    COUNT(*)
FROM events
GROUP BY tableoid
ORDER BY partition_name;
```

Задание

Ответьте:

1. В какую партицию попадёт запись с `created_at = '2026-09-10 12:00:00'`?
 2. В какую партицию попадёт запись с `created_at = '2026-09-11 00:00:00'`?
 3. Что произойдёт, если вставить запись за 2026-09-12?
 4. Почему граница `TO ('2026-09-11')` не включает эту дату?
-

4. Часть 2. Partition pruning

Выполните запрос:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*)
FROM events
WHERE created_at >= '2026-09-10'
AND created_at < '2026-09-11';
```

Изучите план.

Задание

Определите:

- сколько партиций PostgreSQL проверил;
- какая партиция действительно нужна запросу;
- удалось ли PostgreSQL исключить остальные партиции;
- где в плане видно использование partition pruning.

Теперь выполните:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*)
FROM events
WHERE event_type = 'click';
```

Сравните планы.

Вопрос

Почему второй запрос может обращаться ко всем партициям?

5. Часть 3. RANGE PARTITIONING по числовому значению

Партиционирование не обязательно должно выполняться по дате.

Создайте отдельную таблицу:

```
CREATE TABLE products (
    id BIGINT NOT NULL,
    name TEXT NOT NULL,
    price NUMERIC NOT NULL
) PARTITION BY RANGE (price);
```

Создайте:

cheap	< 100
medium	100–999
expensive	1000+

Например:

```
CREATE TABLE products_cheap
PARTITION OF products
FOR VALUES FROM (0) TO (100);
```

```
CREATE TABLE products_medium
PARTITION OF products
FOR VALUES FROM (100) TO (1000);
```

```
CREATE TABLE products_expensive
PARTITION OF products
FOR VALUES FROM (1000) TO (MAXVALUE);
```

Задание

Добавьте товары с разными ценами.

Проверьте, в какие партиции они попали.

Затем выполните:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM products
WHERE price >= 100
      AND price < 500;
```

Ответьте:

Какие партиции PostgreSQL должен просмотреть и почему?

6. Часть 4. LIST PARTITIONING

Теперь рассмотрим ситуацию, когда данные разделяются не по диапазону, а по категориям.

Создайте:

```
CREATE TABLE customers (
    id BIGINT NOT NULL,
    name TEXT NOT NULL,
    customer_type VARCHAR(30) NOT NULL
) PARTITION BY LIST (customer_type);
```

Создайте партиции:

B2C
B2B
Enterprise

Например:

```
CREATE TABLE customers_b2c
PARTITION OF customers
FOR VALUES IN ('B2C');
```

```
CREATE TABLE customers_b2b
PARTITION OF customers
FOR VALUES IN ('B2B');
```

```
CREATE TABLE customers_enterprise
PARTITION OF customers
FOR VALUES IN ('Enterprise');
```

Добавьте данные.

Задание

Выполните:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customers
WHERE customer_type = 'B2B';
```

Определите:

- какая партиция используется;
 - почему PostgreSQL может не обращаться к другим;
 - чем такой сценарий отличается от RANGE.
-

7. Часть 5. LIST PARTITIONING: что делать с неизвестным значением?

Попробуйте добавить:

```
INSERT INTO customers
VALUES (100, 'Test User', 'VIP');
```

Посмотрите, что произошло.

Теперь создайте DEFAULT partition:

```
CREATE TABLE customers_default
PARTITION OF customers DEFAULT;
```

Повторите вставку.

Задание

Объясните:

1. зачем нужна DEFAULT partition;
 2. чем она полезна;
 3. какие проблемы могут возникнуть, если постоянно складывать новые значения в DEFAULT.
-

8. Часть 6. HASH PARTITIONING

Теперь рассмотрим другой сценарий.

Предположим, что данные необходимо равномерно распределить между несколькими партициями.

Создайте:

```
CREATE TABLE user_events (  
    id BIGINT NOT NULL,  
    user_id BIGINT NOT NULL,  
    event_type VARCHAR(50),  
    created_at TIMESTAMP NOT NULL  
) PARTITION BY HASH (user_id);
```

Создайте 4 партиции:

```
CREATE TABLE user_events_0  
PARTITION OF user_events  
FOR VALUES WITH (MODULUS 4, REMAINDER 0);
```

```
CREATE TABLE user_events_1  
PARTITION OF user_events  
FOR VALUES WITH (MODULUS 4, REMAINDER 1);
```

```
CREATE TABLE user_events_2  
PARTITION OF user_events  
FOR VALUES WITH (MODULUS 4, REMAINDER 2);
```

```
CREATE TABLE user_events_3  
PARTITION OF user_events  
FOR VALUES WITH (MODULUS 4, REMAINDER 3);
```

Загрузите большое количество данных.

Проверьте распределение:

```
SELECT  
    tableoid::regclass AS partition_name,  
    COUNT(*)  
FROM user_events  
GROUP BY tableoid  
ORDER BY partition_name;
```

Задание

Ответьте:

1. Насколько равномерно распределились данные?
2. Почему HASH может быть полезен?
3. Чем HASH отличается от RANGE?
4. Почему HASH плохо подходит для сценария «удалить данные старше 3 лет»?

9. Часть 7. Выбор стратегии партиционирования

Теперь небольшая задача на проектирование.

Для каждого сценария выберите подходящую стратегию:

Сценарий А

В систему каждый день поступают миллионы событий.

Через 3 года необходимо быстро удалить данные старше 3 лет.

Что выбрать?

Сценарий В

Есть пользователи трёх типов:

B2C

B2B

Enterprise

Основные запросы фильтруют пользователей по этому признаку.

Что выбрать?

Сценарий С

Необходимо равномерно распределить данные по нескольким физическим разделам по `user_id`.

Что выбрать?

Сценарий D

Есть таблица заказов, и аналитика почти всегда работает с диапазоном:

`created_at BETWEEN ...`

Что выбрать?

Сценарий E

Есть таблица платежей, которую необходимо разделять по странам:

EE

LV

LT

FI

SE

Что выбрать?

Для каждого ответа необходимо написать **одно-два предложения с обоснованием**.

10. Часть 8. Партиционирование и индексы

Партиционирование не отменяет индексы.

Создайте индекс:

```
CREATE INDEX idx_events_user_id  
ON events (user_id);
```

Теперь выполните:

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT *  
FROM events  
WHERE created_at >= '2026-09-10'  
      AND created_at < '2026-09-11'  
      AND user_id = 12345;
```

Задание

Проанализируйте:

1. какие партии были отфильтрованы через partition pruning;
2. используется ли индекс;
3. на каком уровне существует индекс;
4. почему комбинация partitioning + index может быть эффективнее, чем использование только одного механизма.

11. Часть 9. Когда партиционирование не помогает

Рассмотрим запрос:

```
SELECT COUNT(*)  
FROM events  
WHERE event_type = 'click';
```

Выполните:

```
EXPLAIN (ANALYZE, BUFFERS)
```

Задание

Попробуйте оптимизировать этот запрос.

Создайте индекс:

```
CREATE INDEX idx_events_event_type  
ON events (event_type);
```

Повторите EXPLAIN ANALYZE.

Сравните результаты.

Ответьте:

Почему partitioning сам по себе не решил проблему этого запроса?

Сделайте вывод:

Партиционирование отвечает на одну проблему, а индекс — на другую.

12. Часть 10. Автоматическое создание партиций

Теперь переходим от SQL к эксплуатации.

Представим, что таблица:

events

партиционирована по дням.

Сегодня:

2026-09-11

Система должна всегда иметь партиции минимум на **3 дня вперёд**:

2026-09-11

2026-09-12

2026-09-13

2026-09-14

Создавать их вручную нельзя.

Необходимо реализовать job:

CreatePartitionsJob

↓

определить сегодняшнюю дату

↓

определить требуемый горизонт

↓

получить существующие партиции

↓

найти отсутствующие

↓

создать их

Требования

Job должна:

- создавать недостающие партиции;
- не создавать уже существующие;
- быть безопасной при повторном запуске;
- логировать результат работы.

Например:

2026-09-11 01:00

Partition job started.

Existing partitions: 3
Required partitions: 4
Missing partitions: 1

Creating:
events_2026_09_14

Partition created successfully.

Partition job finished.

13. Часть 11. Production-сценарий: ночной job сломался

Теперь представим реальную ситуацию.

Ночью должна была выполняться job:

01:00
CreatePartitionsJob

Но job не отработала.

Например:

events_2026_09_11
events_2026_09_12
events_2026_09_13

есть.

А:

events_2026_09_14

нет.

При этом никто не должен узнавать о проблеме только после того, как приложение начало падать на INSERT.

Задание

Необходимо реализовать **проверку состояния партиций и alerting**.

Создайте отдельную проверку:

PartitionHealthCheck

Она должна определить:

существуют ли все партиции, которые должны существовать на ближайшие 3 дня?

Например:

Сегодня: 11 сентября

Ожидается:

11 сентября	✓
12 сентября	✓
13 сентября	✓
14 сентября	✗

Результат проверки:

CRITICAL

11.1. Отправка уведомления

Если нужной партиции нет, необходимо отправить уведомление.

Можно использовать:

- Vk;
- Max;
- email;
- другой канал.

Сообщение должно содержать:

 Partition alert

Table: events

Missing partitions:
events_2026_09_14

Expected horizon: 3 days

Checked at:
2026-09-11 08:00:00

11.2. Как проверить, что alert действительно работает

Вы должны **искусственно создать ошибку**.

Например:

1. создать несколько будущих партиций;
 2. удалить одну: **DROP TABLE** events_2026_09_14;
 3. запустить PartitionHealthCheck;
 4. убедиться, что проверка обнаружила проблему;
 5. убедиться, что уведомление пришло.
-

11.3. Восстановление

После получения alert вы должны восстановить систему

После исправления повторная проверка должна вернуть:

OK

Например:

 Partition check OK

Table: events

All required partitions exist.

Checked at:
2026-09-11 08:15:00

11.4. Дополнительное задание

Сделайте так, чтобы система не отправляла один и тот же alert бесконечно.

Например:

08:00 – CRITICAL → отправить alert
08:05 – CRITICAL → не отправлять повторно
08:10 – CRITICAL → не отправлять повторно
08:15 – OK → отправить recovery

Таким образом, система должна реагировать не только на ошибку, но и на её восстановление.

14. Часть 12. Партиционирование собственной базы данных

Теперь примените полученные знания к **своему backend-сервису**.

Выберите таблицу, которая:

- содержит большое количество данных или потенциально быстро растёт;
- имеет подходящий ключ для разделения;
- действительно может получить пользу от partitioning.

Наиболее естественный вариант:

`created_at`

Но использовать его **не обязательно**.

Вы можете выбрать:

- дату;
- числовой диапазон;
- категорию;
- статус;
- регион;
- tenant;
- другой подходящий атрибут.

Главное — обосновать решение.

Что необходимо сделать

Шаг 1. Выбрать таблицу

Например:

`orders`

Шаг 2. Выбрать ключ

Например:

`created_at`

Шаг 3. Выбрать стратегию

Например:

`RANGE`

Шаг 4. Объяснить выбор

Необходимо ответить:

Почему именно эта таблица?

Почему именно этот ключ?

Почему именно эта стратегия?

Какие запросы должны получить пользу от partition pruning?

Шаг 5. Создать минимум 3 партиции

Например:

`orders_2026_09`

`orders_2026_10`

`orders_2026_11`

или другой вариант, соответствующий выбранной стратегии.

Шаг 6. Проверить запросы

Выберите минимум **3 реальных запроса** вашего API.

Например:

`GET /orders?from=...&to=...`

`GET /orders/{id}`

`GET /orders/statistics`

Для SQL-запросов используйте:

EXPLAIN (**ANALYZE**, BUFFERS)

Покажите, какие из запросов используют partition pruning.

Шаг 7. Автоматизировать создание/удаление партиций

Если в вашем сервисе данные растут по времени, реализуйте:

nightly job



создание будущих партиций

Если выбран другой тип partitioning, реализуйте соответствующую автоматизацию.

Шаг 8. Добавить контроль

Система должна уметь определить:

партиции существуют → ОК

партиция отсутствует → проблема

Шаг 9. Добавить alert

При обнаружении проблемы отправьте уведомление:

VK / Max / email / другой канал

15. Что показать на защите

На защите необходимо продемонстрировать **сценарий**.

Свой сервис

Показать:

Большая таблица



Выбор ключа



Выбор стратегии



Партиции



Partition pruning



Автоматическое создание



Проверка



Сбой

↓
ALERT
↓
Восстановление
↓
OK

18. Контрольные вопросы

1. Что такое partitioning?
 2. Чем partitioning отличается от индекса?
 3. Какие стратегии partitioning поддерживает PostgreSQL?
 4. Когда использовать RANGE?
 5. Когда использовать LIST?
 6. Когда использовать HASH?
 7. Как выбрать partition key?
 8. Что такое partition pruning?
 9. Почему pruning может не сработать?
 10. Можно ли использовать индексы вместе с partitioning?
 11. Что произойдёт, если подходящей партиции нет?
 12. Зачем создавать будущие партиции заранее?
 13. Почему создание партиций должно быть автоматизировано?
 14. Чем логирование ошибки отличается от alerting?
 15. Что такое recovery alert?
 16. Почему нельзя бесконечно отправлять одинаковый alert?
 17. Всегда ли partitioning ускоряет запросы?
 18. Какие проблемы возникают при неправильном выборе размера партиций?
-

Итог

Партиционирование — это не просто команда:

PARTITION BY RANGE (...)

В production-системе необходимо думать обо всём жизненном цикле:

спроектировать
↓
создать
↓
заполнить
↓
использовать

↓
создавать новые
↓
контролировать
↓
обнаруживать сбои
↓
восстанавливать

Поэтому финальная часть лабораторной должна показать не только то, что вы умеете создавать партиции, но и то, что умеете **эксплуатировать партиционированную систему**.